

Contents

Guide 4 — The Class Diagram (the data model), explained simply	1
1. What the diagram is	1
2. The eight boxes, one line each	1
3. The arrows (relationships) and the “1” and “N”	2
4. Small glossary (the words that look technical)	2
5. Jury questions — ready answers	2

Guide 4 — The Class Diagram (the data model), explained simply

What the class diagram in Chapter 2 means, in plain words: what each box is, what the arrows mean, and a small glossary at the end (including what “slug” means).

1. What the diagram is

The class diagram is just **the list of things the app stores**, drawn as boxes. Each box is one kind of record (one table in the database). A box has **three parts**, top to bottom:

- 1. **Name** — the kind of thing (Operator, Offer, User...).
- 2. **Attributes** — the pieces of data it keeps (an Offer keeps a name, price, data...).
- 3. **Methods** — the things it can *do* in the code (an Offer can check whether it matches the user’s needs).
Every method is a real function in the project, not invented.

The **arrows** between boxes show how the records are linked.

2. The eight boxes, one line each

- **Operator** — a mobile operator (Djezzy, Ooredoo, Mobilis). Stores its name and website; can list its active offers.
 - **Offer** — one mobile plan. Stores name, price, data, validity, type (prepaid / postpaid), and network (4G / 5G); can check whether it matches a user’s needs and whether a value (calls, SMS) is unlimited.
 - **PriceHistory** — one line per price change of an offer (so we can show the price over time). Stores the price and the date it was recorded.
 - **User** — a registered account. Stores name, email, password, and role (user or admin); can verify its password at login and tell whether it is an admin.
 - **UserProfile** — the user’s stated needs, used by the recommendation engine. Stores the monthly budget, the data needed, and the priority ranking; can hand those to the engine.
 - **SavedOffer** — a bookmark: it records that a given user saved a given offer. Stores the date; can be toggled on or off.
 - **Notification** — one alert shown to a user (new offer, price drop, recommendation, or an admin alert). Stores the title, message, type, and whether it was read; can be marked read.
 - **ScrapeLog** — one record per scrape run, for the admin dashboard. Stores the status, how many offers were found, and any error; is marked complete or failed at the end.
-

3. The arrows (relationships) and the “1” and “N”

An arrow is a **link** between two records. The small **1** and **N** say *how many*:

- **Operator 1 — N Offer**: one operator has **many** offers; each offer belongs to **one** operator.
- **Operator 1 — N ScrapeLog**: one operator, **many** scrape runs over time.
- **Offer 1 — N PriceHistory**: one offer, **many** price records over time.
- **User 1 — N Notification**: one user, **many** notifications.
- **User 1 — 1 UserProfile**: each user has **exactly one** profile.
- **User 1 — N SavedOffer** and **Offer 1 — N SavedOffer**: SavedOffer sits in the middle and links the two, so **a user can save many offers and an offer can be saved by many users** (this is the standard way to record a “many to many” link).

So, read aloud: “one operator has many offers; each offer keeps a history of prices; a user has one profile, many notifications, and many saved offers.”

4. Small glossary (the words that look technical)

- **Attribute** — a piece of data a record stores (an Offer’s *price*).
 - **Method (operation)** — something the record can *do*, written as a function in the code (*matchesFilters()* checks if an offer fits the user’s needs).
 - **Association** — the arrow; a link between two records.
 - **1 and N** — how many on each side. **1 — N** = “one to many”; **1 — 1** = “one to one”.
 - **slug** — a short, web-safe version of a name: lowercase, with dashes instead of spaces and accents removed. For example the offer “Djezzy LEGEND 4000” has the slug `djezzy-legend-4000`. It is used inside web addresses and to match a freshly scraped offer back to its existing database row. It is a technical detail the user never sees, which is why it is **left out of the simplified diagram**.
 - **-1 means unlimited, 0 means none** — a convention for calls, SMS, and data, so the app can tell “unlimited” apart from “zero”.
 - **id / primary key** — the unique number that identifies one record. Every box has one; it is omitted from the simplified diagram to keep it readable. The arrows are how one record points to another’s id.
-

5. Jury questions — ready answers

- **Why eight boxes?** Each is one real responsibility: the catalogue (Operator, Offer, PriceHistory, ScrapeLog) and the user side (User, UserProfile, SavedOffer, Notification).
 - **Why is UserProfile separate from User?** A user may exist without ever filling a recommendation profile; keeping the profile in its own box keeps the account simple and lets the profile change without touching the account.
 - **Why is SavedOffer its own box and not just a list on the user?** Because the same offer can be saved by many users and a user can save many offers; a small linking record is the clean way to store that.
 - **The boxes have methods now — are they invented?** No. Each method maps to a real function in the code (for example *verifyPassword()* is the bcrypt check at login, *toNeeds()* turns the profile into the input the recommendation engine reads).
 - **Where is the full detail (every field, every type)?** In the entities table next to the diagram in Chapter 2; the diagram is the simple overview.
-

Companion to the recommendation, scraping, and search guides. Tell me the next concept you want explained.